Mississippi Valley State University

CS 341 — Discrete Structures

Practice Set 45: Approved Solutions

Searching and Sorting

Covers Chartrand & Zhang, Section 6.4

Status of this set

Ungraded practice. Solutions released with it. Show each pass, not just the sorted list.

Part I: How to Practice This Material

The five algorithms at a glance

Algorithm	Complexity	Requires
Linear search	$O(n)$	nothing
Binary search	$O(\log n)$	a sorted list
Bubble sort	$\Theta(n^2)$	nothing
Insertion sort	$O(n^2)$	nothing
Merge sort	$\Theta(n \log n)$	nothing

Three mistakes to expect

1. **Running binary search on an unsorted list.** The halving step is meaningless without order.
2. **Reporting only the final sorted list.** The passes are the answer.
3. **Confusing the merge bound with the merge-sort bound.** Merging two lists costs at most $m + n - 1$; sorting costs $\Theta(n \log n)$.

Part II: Practice Problems

Section A — Searching

1. Use binary search to determine whether 27 appears in $s : 2, 5, 9, 14, 20, 23, 27, 31$. Show each pass.

Approved solution

Pass	Range	Midpoint	Action
1	positions 1–8	14	$27 > 14$, go right
2	positions 5–8	23	$27 > 23$, go right
3	positions 7–8	27	found

Three passes. Linear search would have taken seven comparisons to reach position 7.

2. Repeat for the target 25, which is not in the list. How does the algorithm know to stop?

Approved solution

The first three passes are identical: midpoints 14, 23, 27. On pass 3, $25 < 27$, so the search moves left, leaving the range with its lower bound above its upper bound.
The algorithm stops when the range becomes **empty** — that is, when $a > b$ — and reports that the target is absent.

An absent target costs the same as a worst-case present one. Both run the full $O(\log n)$ passes, since the range must shrink to nothing either way.

Section B — Sorting

3. Sort $s : 9, 3, 7, 1$ by bubble sort, showing the list after each pass. How many comparisons does it make?

Approved solution

Start	9, 3, 7, 1
After pass 1	3, 7, 1, 9
After pass 2	3, 1, 7, 9
After pass 3	1, 3, 7, 9

Comparisons: $3 + 2 + 1 = 6 = \dfrac{4 \cdot 3}{2}$.

Note that after pass 1 the largest element 9 is in place, after pass 2 the next largest is, and so on. That is why each pass is one comparison shorter than the last.

4. Merge the sorted lists $2, 4, 8$ and $1, 3, 5, 9$. How many comparisons does it take, and what does Theorem 6.43 allow?

Approved solution

Result: $1, 2, 3, 4, 5, 8, 9$.
Comparisons: 6. Theorem 6.43 allows at most $m + n - 1 = 3 + 4 - 1 = 6$, so this merge hits the bound exactly.

The bound is reached when the two lists interleave so that neither empties early. If one list is entirely

smaller than the other, the merge finishes sooner.

5. Sort $s : 5, 2, 8, 1, 9, 3, 7, 4$ by merge sort. Show the divide and merge stages.

Approved solution

Divide:

$$5\ 2\ 8\ 1 \mid 9\ 3\ 7\ 4 \longrightarrow 5\ 2 \mid 8\ 1 \mid 9\ 3 \mid 7\ 4 \longrightarrow 5 \mid 2 \mid 8 \mid 1 \mid 9 \mid 3 \mid 7 \mid 4$$

Merge:

To pairs	$2\,5 \mid 1\,8 \mid 3\,9 \mid 4\,7$
To fours	$1\,2\,5\,8 \mid 3\,4\,7\,9$
To eight	$1\,2\,3\,4\,5\,7\,8\,9$

This trace uses 17 comparisons, and Chartrand's bound for $n = 8$ is $n \log n - n + 1 = 24 - 8 + 1 = 17$ — reached exactly here.

There are $\log_2 8 = 3$ merge levels, each handling all 8 elements. That is where $n \log n$ comes from.

Section C — Writing

6. In five to eight complete sentences, explain why merge sort beats bubble sort by a factor that grows with n, and why no faster computer closes the gap.

Approved solution

Model answer. Bubble sort compares every pair of positions, making about $n^2/2$ comparisons regardless of the input, while merge sort splits the list roughly $\log_2 n$ times and does about n units of merging work at each level, for about $n \log n$ in total. The ratio between the two counts is therefore about $n/\log n$, which is not a fixed number but grows as the input grows. At a thousand items merge sort does roughly a hundred times less work; at a million, roughly fifty thousand times less. A faster computer multiplies both running times by the same constant, so it shifts the crossover slightly but leaves the ratio untouched. That is exactly the distinction big-O is built to capture: constants are a property of the machine, and the growth rate is a property of the algorithm.

Full credit requires the observation that the ratio itself grows, which is what makes the gap unclosable by hardware.

Instructor notes

- **Problem 2** shows an absent target costs the same as a worst-case present one. Students often assume failure is cheaper.
- **Problem 4 hits the merge bound exactly**, and Problem 5's merge sort hits Chartrand's bound exactly too. Worth noting that bounds are sometimes tight.
- **Problem 6** is the writing anchor and the argument for the whole unit.
- Reserved for Homework 15: Section 6.4 Exercises 2, 4, 6, and 10 (even).